

Reviewer Pack — Sunflower Petal Count Upper-Bounds Forman-Ricci μ on Random ...

Entry 4aa24a75eefb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:28:46 UTC

Sunflower Petal Count Upper-Bounds Forman-Ricci μ on Random k -DNFs

Entry ID: 4aa24a75eefb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 10:28:46 UTC

1. Conjecture

Field A (mathematical branch): Sunflower lemma (Erdős-Rado kernel/petal structure of k -uniform set families) **Field B** (complexity object): Monotone Boolean functions (combinatorial Forman-Ricci proxy μ on random k -uniform monotone DNFs of size $\leq N^c$)

Statement:

Let $v=2N$, $k=\lceil \log_2 v \rceil$. Sample a random k -uniform monotone DNF F by drawing $|F|=\lfloor N^c \rfloor$ distinct k -subsets of $[v]$ uniformly without replacement. Let $G_F=(E_F)$ be the term-overlap graph with edges $\{T,T'\}$ iff $T \cap T' \neq \emptyset$. Define the combinatorial Forman-Ricci excess $\mu(F) := \max_{\{T,T'\} \in E_F} \max(0, \deg_{G_F}(T) + \deg_{G_F}(T') - 4)$, and the sunflower petal count $\kappa(F) := \max p$ such that F contains a p -sunflower (p terms whose pairwise intersections all equal a common core Y , possibly $Y=\emptyset$). Conjecture: for every $(N,c,\text{seed}) \in \{10,15,20\} \times \{1,1.5\} \times \{1,\dots,30\}$, $\mu(F) \leq 6c \cdot \log_2(1+\kappa(F)) + 4$; a single configuration with $\mu(F) > 6c \cdot \log_2(1+\kappa(F)) + 4$ falsifies it.

Rationale (proposer's reasoning):

SC4 demands $\mu(F) \leq 6c \cdot \log(1+N)$ for $|F| \leq N^c$, and the Erdős-Rado sunflower lemma is the canonical tool for converting a size budget $|F| \leq N^c$ into a structural petal count κ ; if μ is controlled by κ (a coarser quantity than N), the SC4 bound follows because $\kappa \leq |F| \leq N^c$ gives $6c \cdot \log_2(1+\kappa) \leq 6c \cdot \log_2(1+N^c) \leq 6c^2 \cdot \log_2(1+N)$. Sunflower kernels concentrate term-degree mass at a common core, which is precisely the topological mechanism that forces a vertex (term) in G_F to have high degree — so any large μ should be ‘explained’ by a corresponding sunflower. This bridges combinatorial Forman-Ricci to Erdős-Rado structure with no algebrization/relativization risk because the construction is purely set-combinatorial and the predicate is non-polynomial-time-decidable across truth tables (it lives on the DNF, not on the function).

Taxonomy category: forman_ricci_dnf (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5b951fbd1a0d6ad9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 180 configs ($N \in \{10, 15, 20\} \times c \in \{1, 1.5\} \times \text{seed} \in \{1..30\}$), compute slack $s = 6c \cdot \log_2(1 + \kappa(F)) + 4 - \mu(F)$. Conjecture is SUPPORTED iff $\min_{\text{seed}} s \geq 0$ (i.e., every single config satisfies $\mu \leq 6c \cdot \log_2(1 + \kappa) + 4$); FALSIFIED iff any single config has $s < 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - sunflower lemma monotone DNF random k-uniform - Forman-Ricci curvature combinatorial hypergraph intersection - Erdos-Rado sunflower petal random k-DNF term overlap graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    N = 10 if seed % 2 == 0 else 15 if seed % 3 == 0 else 20
    c = 1 + (seed // 60) % 2
    v = 2 * N
    k = math.ceil(math.log2(v))

    rng = random.Random(seed)
    F = set(rng.sample(range(1, v+1), int(N*c)))

    G_F = {}
    for T in F:
        for T_prime in F:
            if T != T_prime and len(T & T_prime) > 0:
                G_F.setdefault(T, []).append(T_prime)
                G_F.setdefault(T_prime, []).append(T)

    def degree(node):
        return len(G_F.get(node, []))

    mu = max(max(0, degree(T) + degree(T_prime) - 4) for T in F for T_prime in G_F[T] if T != T_prime)
```

```

sunflowers = {}
for T in F:
    for T_prime in G_F[T]:
        core = T & T_prime
        if core not in sunflowers:
            sunflowers[core] = []
            sunflowers[core].append((T, T_prime))

κ = max(len([T for T, T_prime in pairs if len(T & T_prime) == len(core)]) for core, pairs in sunflowers.items())

s = 6 * c * math.log2(1 + κ) + 4 - μ
conjecture_holds = s >= 0

return {
    "metric_name": "slack",
    "metric_value": s,
    "instances_tested": len(F),
    "n_max": N,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"μ={μ}, 6c·log₂(1+κ) + 4 = {6 * c * math.log2(1 + κ) + 4 - μ}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(3, 8)] # Default to first 7 seeds
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05a19f51.py", line 66, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05a19f51.py", line 30, in run_trial

```

```
if T != T_prime and len(T & T_prime) > 0:
    ^^^^^^^^^^^^^^^^^
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` before producing any data, indicating a bug in the trial code (treating an int as a set when computing intersections). No configurations were evaluated, so the pre-registered support condition cannot be verified. | next: Fix the term representation in `run_trial` so each term `T` is stored as a frozenset of `k` vertices (not an int), then rerun all 180 (`N,c,seed`) configs and report min slack `s`.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	119212
2	propose	claude_max	opus	0	0	69790
3	propose	claude_max	opus	0	0	70491
4	propose	claude_max	opus	0	0	72996
5	preregistration	claude_max	opus	0	0	6043
6	novelty	claude_max	opus	0	0	3614
7	novelty	claude_max	opus	0	0	5306
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15409
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10341
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10732
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9099
12	judge	claude_max	opus	0	0	4369

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 397403 ms total latency. Provider mix: {'claude_max': 8, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/4aa24a75eefb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4aa24a75eefb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4aa24a75eefb.tar.gz (if generated)